When a problem requires you to use network flow techniques, whether on this problem set or on an exam, you need to do the following. These are required unless otherwise explicitly stated. Each problem on this problem set that asks for an algorithm requires network flow techniques.

- Create the graph if necessary. If a graph was provided in a problem (such as at least one problem below), if you modify it, make the modification very clear.
  - If you create the graph, you must specify the vertices, the edges, the source, the sink, and the capacities clearly. If the grader cannot distinguish this on a test, your score may be zero, even if some other elements would otherwise be okay.
- Specify where within your algorithm you run network flow. By default, we will assume you are running the scaling variant from lecture unless you specify otherwise. If you are reading this before we cover that in lecture, assume there is an algorithm for network flow that runs in time polynomial in the **size** of the graph.
- After the call to network flow, explain how you extract your solution. For example, in the tennis problem during lecture, any flow on an edge that was in the original graph meant the two people represented by the endpoints were going to play against one another.
- A solution whose running time is not a polynomial function **of the original input size** will earn zero credit, regardless of other considerations. You must justify the running time by demonstrating that the *size* of the graph you create is a polynomial function of the size of the original input.

. . . . . .

1. When we run the Ford-Fulkerson algorithm to find a minimum-cut, all vertices either end up *source-side* (in the same partition as s) or *sink-side* (in the same partition as t). However, it isn't clear that all vertices in a graph must always end up in the same set.

   Using only $\mathcal{O}(1)$ calls to a network flow algorithm, can you efficiently label all vertices in a graph with one of the following three labels:

   - Always source-side
   - Always sink-side
   - Sometimes one, sometimes the other

   Explain why your algorithm works.

2. Suppose a concert has just ended and C cars are parked at the venue. We would like to determine how long it takes for all of them to leave the area. For this problem, we are given a graph representing the road network where all cars start at a particular vertex s (the parking lot) and several vertices $(t_1, t_2, \ldots, t_k)$ are designated as exits. We are also given capacities (in cars per minute) for each road (directed edges). **Use network flow techniques** to devise an algorithm to determine the amount of time necessary to get all cars out of the area.

© CompSci 161 Fall 2025– Michael Shindler– University of California, Irvine
This Problem Set may not be reposted without the express written permission of the professor teaching this course.

3. Consider the problem faced by a hospital that is trying to evaluate whether its blood supply is sufficient. The basic rule for blood donation is the following. A person's own blood supply has certain *antigens* present; a person cannot receive blood with a particular antigen if their own blood does not have this antigen present. Concretely, blood is divided into four types: A, B, AB, and O. Patients with type A can receive blood types A or O; patients with type B can receive only B or O; patients with type O can receive only O, and patients with type AB can receive any of the four.

   (a) Let s_O, s_A, s_B, s_{AB} denote the supply in whole units of the different blood types on hand. Assume that the hospital knows the projected demand for each blood type d_O, d_A, d_B, D_{AB} for the coming week. Use network flow techniques to devise algorithm to evaluate if the blood on hand will suffice for the projected need.

   (b) Consider the following example:

Blood Type	Supply	Demand
O	50	45
A	36	42
B	11	8
AB	8	3

   Is the 105 units of blood on hand enough to satisfy the 98 units of demand? Find an allocation that satisfies the maximum possible number of patients. Use an argument based on a minimum capacity cut to show why not all patients can receive blood.

   (c) Now, provide an explanation for this fact understandable to the clinic administrators, who have not taken any Computer Science classes. Your explanation should not involve the words *flow, cut, graph* in the sense we use them in this class.

4. Suppose a city has a dog-owning ordinance so that it allows residents to own a maximum of three dogs per household, but restricts each resident to own at most one dog of any given breed (golden retriever, terrier, golden doodle, etc). Describe an efficient algorithm for assigning puppies to residents that provides for the maximum number of puppy adoptions possible while satisfying the constraints that each resident will only adopt puppies that he or she likes, that no resident can adopt more than three puppies, and that no resident will adopt more than one dog of any given breed.

5. *Ad hoc networks*, made up of low-powered wireless devices, have been proposed for situations like natural disasters in which the coordinators of a rescue effort might want to monitor conditions in a hard-to-reach area. The idea is that a large collection of these wireless devices could be dropped into such an area from an airplane and then configured into a functioning network.

   Note that we're talking about (a) relatively inexpensive devices that are (b) being dropped from an airplane into (c) dangerous territory; and for the combination of reasons (a), (b), and (c), it becomes necessary to include provisions for dealing with the failure of a reasonable number of the nodes.

   We'd like it to be the case that if one of the devices v detects that it is in danger of failing, it should transmit a representation of its current state to some other device in the network. Each device has a limited transmitting range – it cannot communicate with other devices

© CompSci 161 Fall 2025– Michael Shindler– University of California, Irvine
This Problem Set may not be reposted without the express written permission of the professor teaching this course.

further than d yards from it (for some numerical parameter d). Moreover, since we don't want it to try transmitting its state to a device that has already failed, we should include some redundancy: a device v should have a set of k other devices that it can potentially contact, each within d yards of it. We'll call this the *back-up set* for device v. Furthermore, to avoid a small number of points-of-failure, we constrain each device to be in the back-up set of at most b other devices.

Suppose you're given a set of n wireless devices and pairwise distances between each of them. **Use network flow techniques** to design a polynomial-time algorithm that determines whether it is possible to choose a back-up set for each device (i.e., k other devices, each within d yards, with no device in the back-up set for more than b other devices).

6. Professor Jones has determined that x priceless artifacts (which belong in a museum) are located in a labyrinth. The labyrinth can be thought of as a graph, with each edge representing a path and each node an intersection of paths. All of the artifacts are in the same treasure room, which is located at one of the intersections. However, the artifacts are extremely burdensome, so Jones can only carry one artifact at a time. There is only one entrance to the labyrinth, which is also a node in the graph. The entrance serves as the only exit as well. All the paths are protected by human-eating vines, which will be woken up after someone passes the path, so Jones can only go through each path once.

   To obtain a single artifact, Jones would have to find a tour which starts at the entrance, reaches the artifact and leads back again, without getting Jones captured by the vines. If he tries to get another artifact, he must find another tour which does not use the edges he's already traversed. Given the map of the labyrinth and the location of the treasure, give a polynomial-time algorithm that determines how many artifacts Jones can obtain (and how to do so). Justify your algorithm.

7. To celebrate Thanksgiving, UCI's Tryptophan Appreciation Club is going to hold a turkey-themed Poetry Festival all day for three days later this quarter[1]. Several poetry sessions have been planned, and a large number of poets have been invited to read their work during the event. You need to select presenting poets, subject to the following constraints:

   - Exactly k poetry readings must be performed each day, and thus $3k$ altogether.
   - Each reading will be one poet's work in one particular type of poetry (limericks, haiku, ballad, vogon, etc).
   - Each type of poem may be presented at most once per day.
   - Each candidate poet has given you a list of types of poetry they are willing to present.
   - Each poet can read for at most three presentations during the entire event.

   Suppose you have a list of the n candidate poets and g different types of poems. You are also given k, the number of readings each day. **Use network flow techniques** to design a polynomial-time algorithm that either assigns a poet and a style to each of the $3k$ readings, or correctly reports that no such assignment is possible.

[1] To the best of my knowledge, not really.

© CompSci 161 Fall 2025– Michael Shindler– University of California, Irvine
This Problem Set may not be reposted without the express written permission of the professor teaching this course.

8. You know those Russian Nesting Dolls? They are spherical but can stand up, are painted to look like people, and can open at the waist to allow a smaller doll to be stored inside of it. Personally, I think they are creepy and they remind me of Chucky.

   I was recently given n dolls capable of nesting. For each ordered pair of dolls (i, j), I know whether or not doll i can be nested into doll j. Because these were a gift, I have to display them on a shelf. Because they are creepy, I want to minimize how many I have to see : fortunately, the nesting property helps with this.

   Use Network Flow techniques to give a polynomial time algorithm that determines the minimum number of nesting dolls I need to have displayed, while having each doll either on display or inside of one on display.

   For partial credit (4/5), determine instead of I can do this while displaying at most k, for some integer k.

9. Consider the problem of Independent Set: Given a graph G, select some subset of the vertices V' such that no two vertices in V' share an edge. Use network flow techniques to give a polynomial time algorithm to find the largest V' for a given input *bipartite graph* G.

   Note: in the past, many considered this the hardest problem on this problem set. That shouldn't scare you away, but don't be discouraged if you don't get it right away. It's still a great problem to help you learn!

Textbook Problems

If you need more practice, consider these problems.

Goodrich & Tamassia textbook, Chapter 16, A-16.1 through A-16.7.

Erickson book, Chapter 11, problems 4, 5, 6, 7, 10, 12, 13, 15, 18

© CompSci 161 Fall 2025– Michael Shindler– University of California, Irvine
This Problem Set may not be reposted without the express written permission of the professor teaching this course.